구현과제 2 보고서

20221867 이채은

[Python 소스 코드 설명]

-프로그램 전체의 흐름은 주석을 통해 설명

```
# Parser 클래스 정의
class Parser :
    def __init__(self) :
        # 선언된 변수(var)와 그에 해당하는 값(value)을 딕셔너리 형태로 저장
        self.vars_and_values = {}

        # <relop> → == | != | < | > | <= | >=
        self.relops = ['==', '!=', '<', '>', '<=', '>=']

        # 최종 결과를 출력하기 위한 answer 리스트
        self.answers = []

    # 문자열 길이가 10 자 이하이며, 소문자 알파벳으로만 구성되어 있는지 확인
    def is_var_true(self, string) :
        return len(string) <= 10 and all(char.islower() for char in string)

    # 문자열 길이가 10 자 이하이며, 숫자로만 구성되어 있는지 확인
    def is_num_true(self, string) :
        return len(string) <= 10 and string.isdigit()

    # 다음에 올 token 과 기대하는 토큰이 일치하는지 확인하고 맞으면
인덱스+1 하는 함수
    def match(self, expected) :
        if self.code[self.index] == expected :
            self.index += 1
```

```
            else :
                self.answers.clear()
                self.index = len(self.code)
                raise SyntaxError('Syntax Error!!')

    # match 함수에서 현재 인덱스가 마지막 인덱스일 예정일 경우 사용할 함수
    def last_match(self, expected) :
        try :
            if self.code[self.index] != expected :
                self.answers.clear()
                self.index = len(self.code)
                raise SyntaxError('Syntax Error!!')
        # 맨 마지막에 ;이 존재하지 않을 경우, 그 전의 계산 결과를 다 지우고 에러 메시지 출력
        except IndexError :
            self.answers.clear()
            self.index = len(self.code)
            raise SyntaxError('Syntax Error!!')

    # 공백 단위로 토큰을 리스트에 넣음, 필드에 들어갈 내용 초기화
    def parse(self, code) :
        # parse 를 하기 전에 이전 기록 지우기
        self.answers.clear()
        self.vars_and_values.clear()
        self.code = code.split()
        self.index = 0
        self.code_len = len(self.code)-1
        #
        # print(self.code)
        try : # try1: program 을 시작
```

```
                self.program()
                try : # try2:  최종  계산  결과  출력
                        if self.answers:
                                result = ['TRUE' if item is True else 'FALSE' if item is False
else int(item) for item in self.answers]
                        print(*result)
                except :
                        pass
        except SyntaxError as e : # program 에서  문제  발생  시,  에러  메시지  출력
                print(e)

    # <program> -> {<declaration>} {<statement>}
    def program(self) :
        #
        # print('program')
        while self.index < len(self.code) :
                if self.code[self.index] == 'int' :
                        self.index += 1
                        self.declaration() # declaration  실행  부분
                elif self.code[self.index] == 'print' or self.code[self.index] == 'do' or
self.is_var_true(self.code[self.index]) == True :
                        self.statement() # statement  실행  부분
                else :
                        self.answers.clear()
                        self.index = len(self.code)
                        raise SyntaxError('Syntax Error!!')

    # <declaration> → <type> <var> ;
    # <type> → int
    def declaration(self) :
```

```
        #
        # print('declaration')
        var_name = self.code[self.index]
        if self.is_var_true(var_name) == False : #  중복  선언  가능
                self.answers.clear()
                self.index = len(self.code)
        self.vars_and_values[var_name] = 0
        self.index += 1
        if self.index < self.code_len :
                self.match(";")
        else :
                self.last_match(";")
                self.index += 1


'''
<statement> → <var> = <aexpr> ; |
                        print <bexpr> ; |
                        print <aexpr> ; |
                        do ' { ' {<statement>} ' } ' while ( <bexpr> ) ;
'''
def statement(self) :
        #
        # print('statement')
        token = self.code[self.index]
        # print 일  경우
        if token == 'print' :
                self.index += 1
                self.expected = self.code[self.index]
                if self.expected in self.relops :
                        self.answers.append(self.bexpr())
```

```
                elif self.is_num_true(self.expected) or self.is_var_true(self.expected) or
self.expected == '(' :
                        self.answers.append(self.aexpr())
                else :
                        self.answers.clear()
                        self.index = len(self.code)
                        raise SyntaxError('Syntax Error!!')
                if self.index < self.code_len :
                        self.match(';')
                        if self.code[self.index] == 'print' or self.code[self.index] == 'do' or
(self.code[self.index] != 'int' and self.is_var_true(self.code[self.index])) == True or
(self.code[self.index] == '}' and '{' in self.code) :
                                pass
                        else :
                                self.answers.clear()
                                self.index = len(self.code)
                                raise SyntaxError('Syntax Error!!')
                else :
                        self.last_match(';')
                        self.index += 1
            # do 일  경우
            elif token == 'do' :
                #
                # print('do')
                self.index += 1
                self.match('{')
                # do  시작  부분
                while self.code[self.index] != '}' :
                        self.statement()
                self.index += 1
```

```
                self.match('while')
                self.match('(')
                first_condition = self.bexpr()
                # while 시작 부분
                while first_condition :
                        #
                        # print('while')
                        self.index = self.code.index('{') + 1
                        while self.code[self.index] != '}' :
                                self.statement()
                        self.index += 1
                        self.match('while')
                        self.match('(')
                        first_condition = self.bexpr()
                self.match(')')
                if self.index < self.code_len :
                                self.match(';')
                                if self.code[self.index] == 'print' or self.code[self.index] == 'do'
or (self.code[self.index] != 'int' and self.is_var_true(self.code[self.index])) == True or
(self.code[self.index] == '}' and '{' in self.code) :
                                        pass
                                else :
                                        self.answers.clear()
                                        self.index = len(self.code)
                                        raise SyntaxError('Syntax Error!!')
                else :
                        self.last_match(';')
                        self.index += 1
            # var 일 경우
```

```
            elif token in self.vars_and_values and self.is_var_true(token) == True : #  이미 선언된  var 만  사용  가능
                    self.index += 1
                    self.match('=')
                    value = self.aexpr()
                    self.vars_and_values[token] = value
                    if self.index < self.code_len :
                            self.match(';')
                            if self.code[self.index] == 'print' or self.code[self.index] == 'do' or (self.code[self.index] != 'int' and self.is_var_true(self.code[self.index])) == True or (self.code[self.index] == '}' and '{' in self.code) :
                                    pass
                            else :
                                    self.answers.clear()
                                    self.index = len(self.code)
                                    raise SyntaxError('Syntax Error!!')
                    else :
                            self.last_match(';')
                            self.index += 1
            else :
                    self.answers.clear()
                    self.index = len(self.code)
                    raise SyntaxError('Syntax Error!!')

    # <bexpr> → <relop> <aexpr> <aexpr>
    def bexpr(self) :
            #
            # print('bexpr')
            op = self.code[self.index]
            self.index += 1
```

```
            left = self.aexpr()
            right = self.aexpr()
            if op == '==':
                return left == right
            elif op == '!=':
                return left != right
            elif op == '<':
                return left < right
            elif op == '>':
                return left > right
            elif op == '<=':
                return left <= right
            elif op == '>=':
                return left >= right
            else :
                self.answers.clear()
                self.index = len(self.code)
                raise SyntaxError('Syntax Error!!')

    # <aexpr> → <term> {( + | - | * | / ) <term>}
    def aexpr(self) :
        #
        # print('aexpr')
        result = self.term()
        while self.index < len(self.code) and self.code[self.index] in ('+', '-', '*', '/') :
            op = self.code[self.index]
            self.index += 1
            term = self.term()
            if op == '+' :
                result += term
```

```
                elif op == '-' :
                        result -= term
                elif op == '*' :
                        result *= term
                elif op == '/' :
                        result /= term
        return result

# <term> → <number> | <var> | ( <aexpr> )
def term(self) :
        #
        # print('term')
        token = self.code[self.index]
        #
        # print('current term: ' + token)
        self.index += 1
        # number 인  경우
        if token.isdigit() :
                return int(token)
        #  이미  선언된  var 인  경우
        elif token in self.vars_and_values :
                return int(self.vars_and_values[token])
        # (aexpr)인  경우
        elif token == '(' :
                result = self.aexpr()
                self.match(')')
                return result
        else :
                self.answers.clear()
                self.index = len(self.code)
```

```
                raise SyntaxError('Syntax Error!!')


# Main 함수
if __name__ == "__main__" :
    parser = Parser()
    while True:
        code = input() # 공백 Enter 는 에러가 아님
        if code == 'terminate' : # terminate 입력하면 프로그램 종료
            break
        parser.parse(code) # Parser 작동 시작
```

[핵심 기능 설명]

```
parse(self, code)
```

: 공백 단위로 토큰을 리스트에 넣어 저장 및 인덱스, 길이 등을 따로 클래스 변수로 저장, 이전 계산 결과는 저장할 필요 없으므로 초기화 진행, try 문을 중첩하여 **1. <program> 시작**과 **2. 최종 계산 결과를 출력하는 부분**으로 분류, **1** 에서 문제 발생 시, 에러 메시지 출력

```
program(self)
```

: <program> -> {<declaration>} {<statement>} 부분을 구현

```
declaration(self)
```

: <declaration> → <type> <var> ; 부분을 구현, 이 문법에서는 변수 중복 선언이 가능하므로 문법에 맞게 제대로 동작하도록 처리함

```
statement(self)
```

: <statement> → ~ 이 이후의 내용을 다 처리함, 입력 경우를 크게 **1. print 인 경우**, **2. do 인 경우**, **3. var 인 경우로** 나눠서 처리, <statement> 뒤에 <declaration>이 나오면 에러 메시지 출력

```
bexpr(self)
aexpr(self)
```

```
term(self)
```

: <bexpr> → <relop> <aexpr> <aexpr> & <aexpr> → <term> {(+ | - | * | /) <term>} & <term> → <number> | <var> | (<aexpr>) 남은 문법들 처리

```
# Main 함수
if __name__ == "__main__" :
    parser = Parser()
    while True:
        code = input() # 공백 Enter는 에러가 아님
        if code == 'terminate' : # terminate 입력하면 프로그램 종료
            break
        parser.parse(code) # Parser 작동 시작
```

: 파서 작동 및 terminate 입력하면 프로그램 종료 처리

-테스트했던 방법

계산 과정을 확인하기 위해 아래와 같이 테스트	계산 결과를 확인하기 위해 에러마다 번호를 부여

`def statement(self) :`
`  #`
`  print('statement')`
	`else :`
	`  self.answers.clear()`
	`  self.index = len(self.code)`
	`  print('6Syntax Error!!')`

[실행 결과 스냅샷]

-IDLE 에서 실행

```
==================== RESTART: /Users/nano/Desktop/Parser.py ====================
int a ; int a ;
int a ; print     a    ;
0
int variable ; variable = 365 ;
ab = 12 ;
Syntax Error!!
float ab ;
Syntax Error!!
int k = 2 ;
Syntax Error!!
int k ; int j ; k = 3 ; j = 20 ; print k + j
Syntax Error!!
int k ; int j ; k = 3 ; j = 20 ; int a ; print k + j ;
Syntax Error!!
int k ; int j ; k = 3 ; j = 20 ; print k + j ;
23
int k ; print k + 100 * 3 / 2 - 1 ;
149
    int x ; x = 10 + 5 - ( 2 + 3 - 5 * 10 ) ; print x ;
    15
    int x ; x = 10 + 5 * 2 ; print x ; print > 10 x ;
    30 FALSE
    int k ; int j ; k = 3 ; j = 20 ; do { print k + ( j - 1 ) * 10 ; k = k - 1 ; } while ( > k + 10 10 ) ; print == k 0 ;
    220 210 200 TRUE
    int i ; int j ; i = 0 ; j = 0 ; do { j = j + i ; i = i + 1 ; } while ( < i * ( 5 - 4 ) 5 ) ; print i ; print j ;
    5 10
    int a ; print a ; a = 34 ;
    0
    terminate
>>>
```

-마지막 샷은 종료되었음을 확인할 수 있도록 크게 캡처